

STRING HANDLING METHODS, EXCEPTION HANDLING, AND OBJECT-ORIENTED CONCEPTS



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

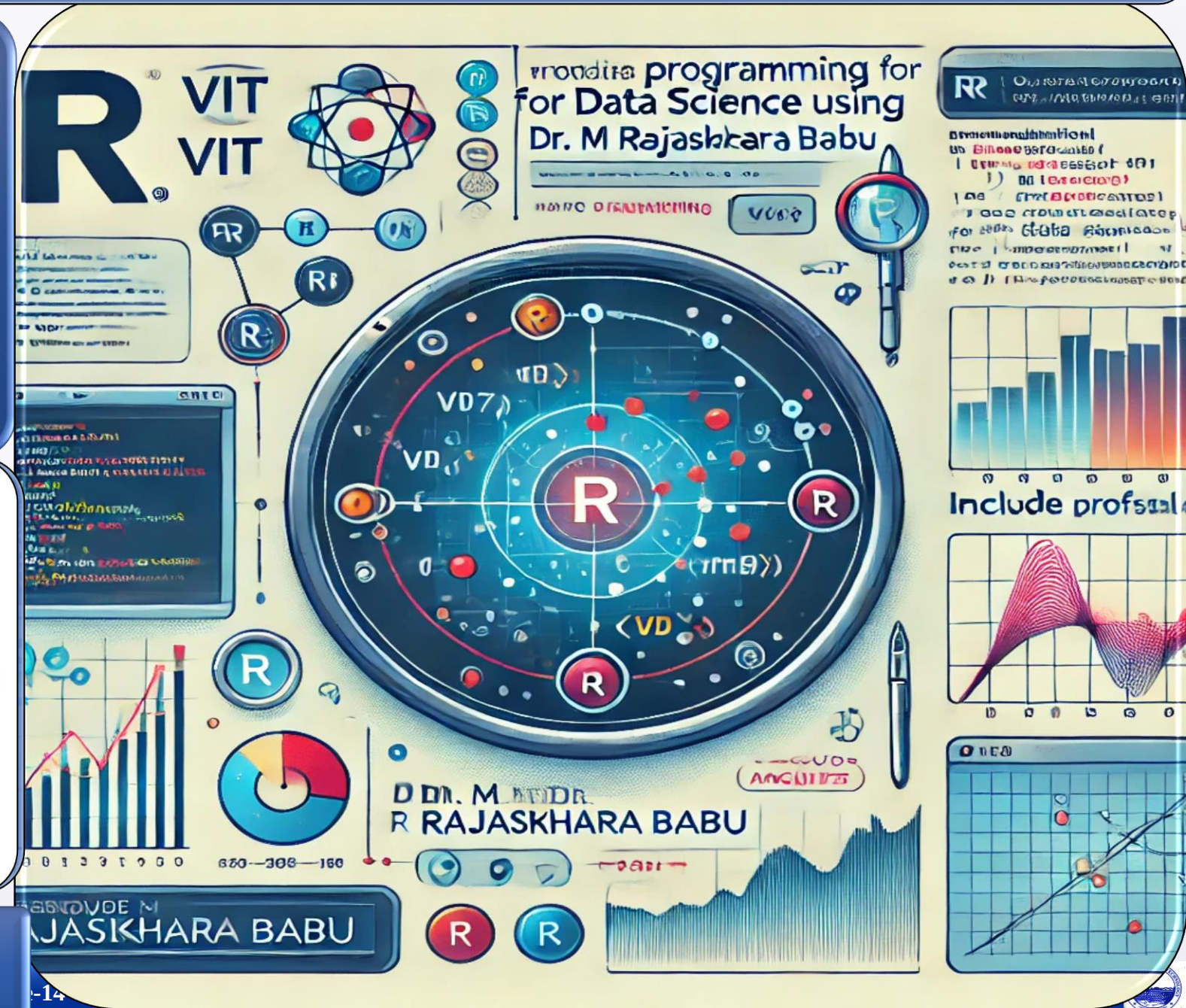


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



Objectives & Teaching Learning Material



To explain string handling techniques and built-in string methods for effective text processing in Python



To demonstrate exception handling mechanisms using try, except, else, and finally blocks for developing robust programs



To introduce object-oriented programming concepts including classes, objects, encapsulation, inheritance, polymorphism, and abstraction

Teaching + Learning

Material:

LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:



Apply Python string methods and formatting techniques to manipulate and process textual data effectively



Develop reliable Python programs by implementing appropriate exception handling mechanisms for runtime errors



Design and implement object-oriented solutions using classes, objects, encapsulation, inheritance, polymorphism, and abstraction



Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
12	String Fundamentals and String Handling Methods	Presentation	Presentation	Listens	Remembering
10	Exception Handling in Python	Explains	Explains	Notes	Understanding
13	Object-Oriented Programming Basics	Case Study	Organizes	Observes	Applying
10	Advanced OOP Concepts and Applications	Discussion	Facilitates	Answers	Analyzing



Part A: String Handling Methods

What We'll Cover

- String basics & characteristics
- Indexing & slicing
- Common string methods
- String formatting

Why It Matters

- Strings are one of the most frequently used data types in Python.
- Mastering string manipulation is essential for data cleaning, text processing, and building real-world applications.



What is a String?

A string is a sequence of characters enclosed within single or double quotes.

Definition & Examples

```
name = "Python"  
course = "Programming"  
msg = 'Hello World'
```

Key Characteristics

✓ **Ordered**

Characters maintain their position.

✓ **Immutable**

Cannot be changed after creation.

✓ **Indexed**

Each character has a numeric index.

✓ **Iterable**

Can be looped over character by character.

Indexing — Access Individual Characters Slicing — Extract a Substring

```
text = "Python"
print(text[0])  # Output: P
print(text[-1]) # Output: n
                (negative index)
```

Positive indices count from the left (starting at 0); negative indices count from the right (starting at -1).

```
text = "Programming"
print(text[0:6])
# Output: Progra
```

Syntax: `string[start:end]` — extracts characters from *start* up to (but not including) *end*.

Common String Methods

Method	Description	Example Output
<code>upper()</code>	Converts to uppercase	<code>"python" → "PYTHON"</code>
<code>lower()</code>	Converts to lowercase	<code>"PYTHON" → "python"</code>
<code>title()</code>	Title case each word	<code>"hello world" → "Hello World"</code>
<code>strip()</code>	Removes leading/trailing spaces	<code>" hi " → "hi"</code>
<code>replace()</code>	Replaces a substring	<code>"Java" → "Python"</code>
<code>split()</code>	Splits into a list	<code>["Python", "Data"]</code>
<code>join()</code>	Joins a list into a string	<code>["a", "b"] → "a b"</code>

replace() — Swap Substrings

```
text = "I like Java"
new_text = text.replace("Java",
"Python")
print(new_text) # I like Python
```

split() — Break into List

```
text = "Python Data Science"
words = text.split()
print(words) #
['Python','Data','Science']
```

join() — Merge a List

```
words =
['Python','Data','Science']
text = " ".join(words)
print(text) # Python Data
Science
```

find() & count() — Search

```
text = "Python"
print(text.find("t")) # 2
print(text.count("o")) # 1
```

f-String Syntax

```
name = "Alice"  
age  = 22  
print(f"{name} is {age} years old")  
# Output: Alice is 22 years old
```

f-Strings (formatted string literals) embed expressions directly inside curly braces {}, making output clean and readable.

Practice Example

```
text = " Data Science "  
print(text.strip())  
# Data Science  
print(text.upper())  
# DATA SCIENCE  
print(text.replace("Science", "Analyti  
cs"))  
# Data Analytics
```

Part B: Exception Handling

An exception is a runtime error that interrupts normal program execution. Without handling, the program terminates abruptly.

ZeroDivisionError

Dividing a number by zero.

ValueError

Invalid input type or value.

FileNotFoundError

Accessing a missing file.

IndexError

Accessing an out-of-range index.



Without Exception Handling

```
num = 10
print(num / 0)

# ZeroDivisionError — Program
Terminates!
```

The program crashes immediately with no user-friendly message.

With Try-Except Block

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
# Output: Cannot divide by zero
```

The error is caught and handled — the program continues running.

Multiple Exceptions

```
try:
    number =
int(input())

except ValueError:
    print("Invalid
Number")

except
ZeroDivisionError:
    print("Division
Error")
```

Else Block

```
try:
    num =
int(input())

except:
    print("Error")

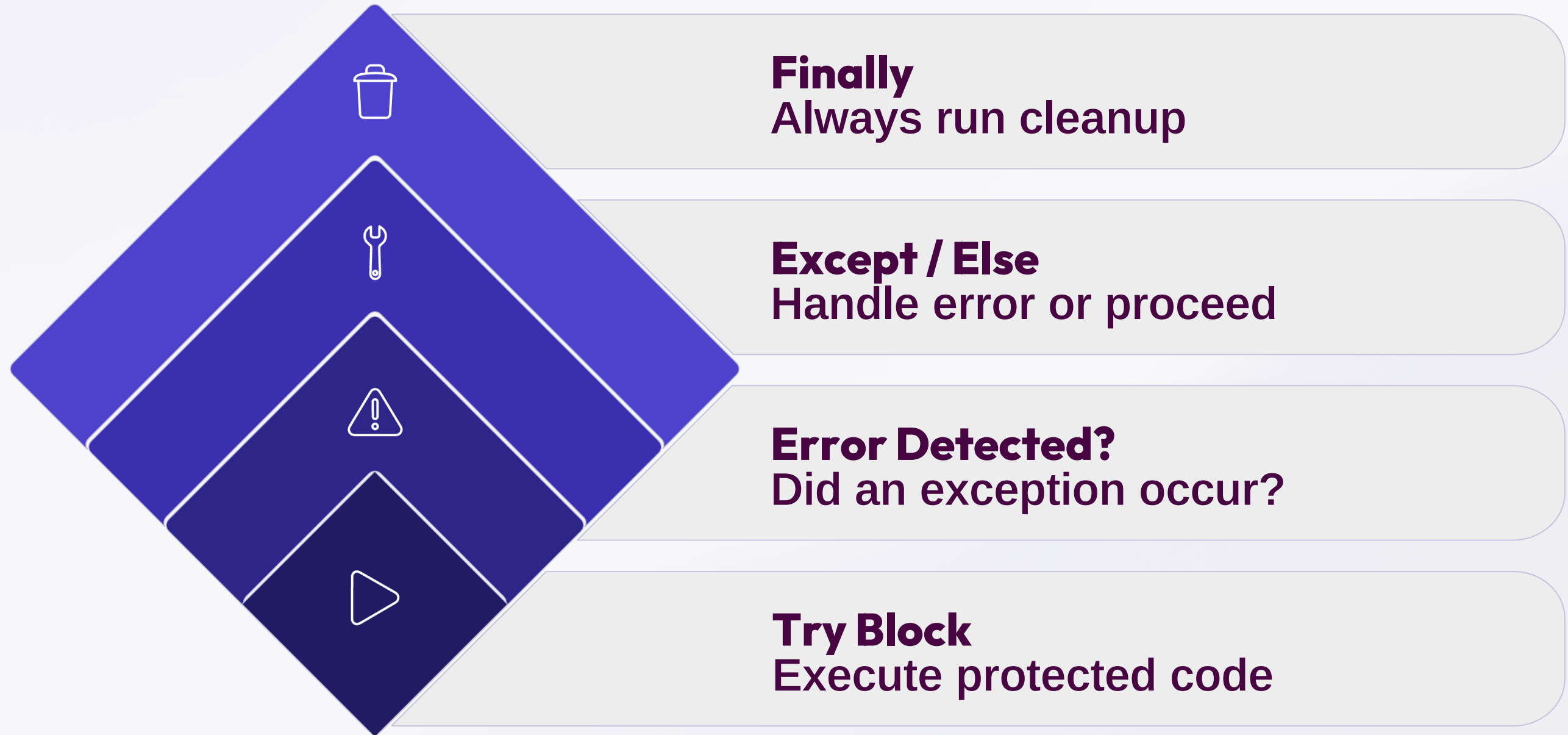
else:
    print("Valid
Input") # Runs
only if no error
```

Finally Block

```
try:
    print("Program")

finally:
    print("Closing
Program") #
Always executes
```

Exception Handling Flow



The try block runs first. If an error occurs, the except block handles it; otherwise, the else block executes. The finally block always runs regardless of outcome — ideal for cleanup tasks like closing files or connections.

Complete Exception Handling Example

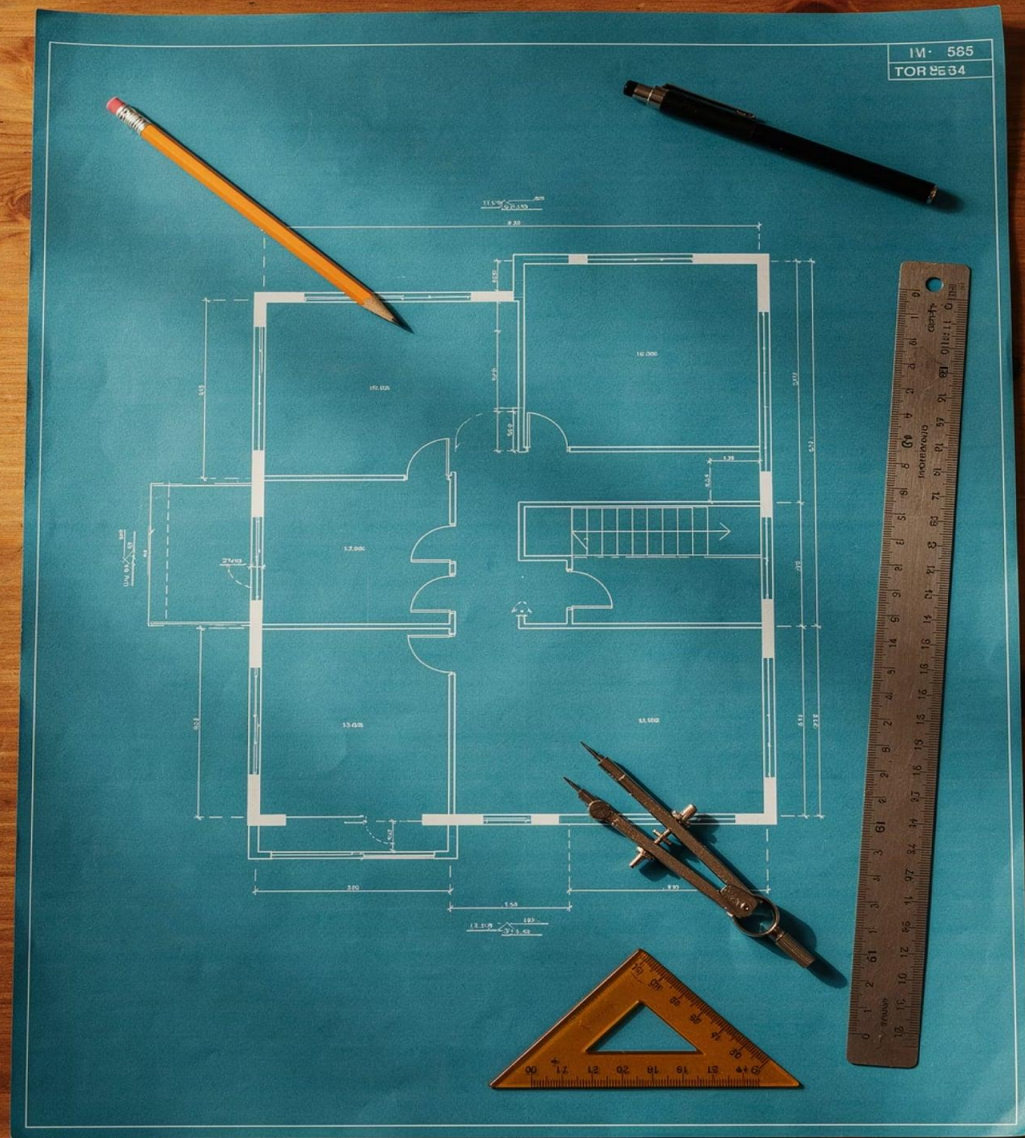
Full Program

```
try:
    a = int(input("Enter A: "))
    b = int(input("Enter B: "))
    result = a / b
except ZeroDivisionError:
    print("Division by zero")

except ValueError:
    print("Invalid Input")
else:
    print("Result =", result)
finally:
    print("Program Ended")
```

What Each Block Does

- 1 try**
Attempts to read two integers and divide them.
- 2 except**
Catches division-by-zero or invalid input errors separately.
- 3 else**
Prints the result only when no exception occurs.
- 4 finally**
Always prints "Program Ended" as a cleanup step.



Object-Oriented Programming (OOP) is a methodology that organizes code around **objects and classes** rather than functions and logic alone.

✓ **Reusability**

Write once, use many times.

✓ **Maintainability**

Easier to update and debug.

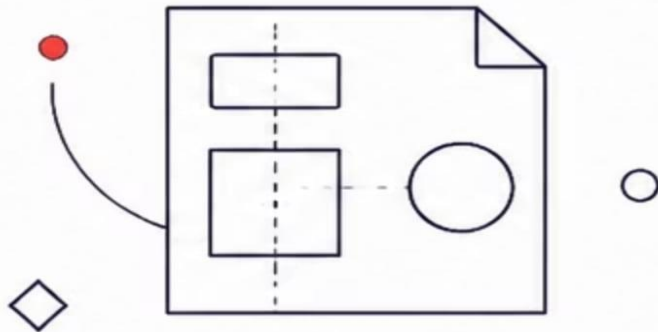
✓ **Modularity**

Break code into logical units.

✓ **Scalability**

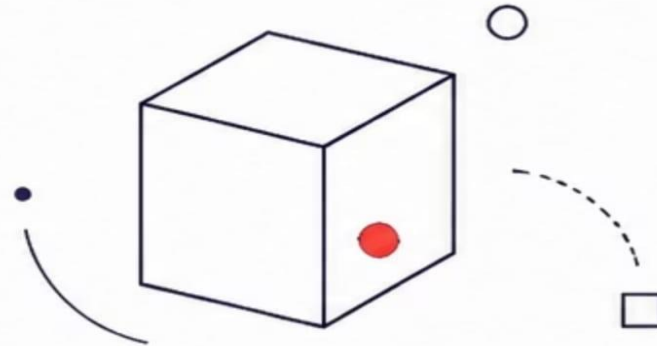
Grows with your application.

Core OOP Concepts: The Four Pillars



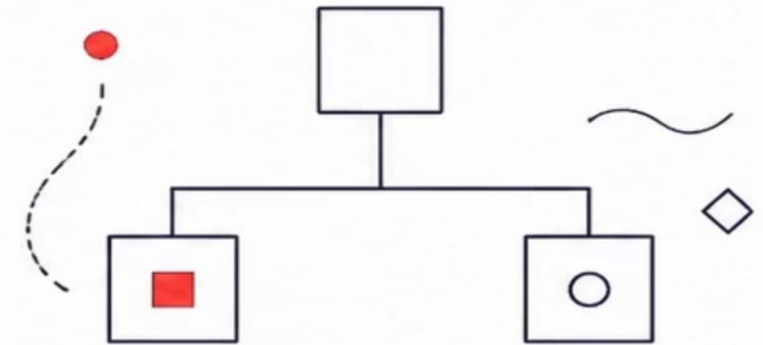
1. Class

(blueprint for creating objects)



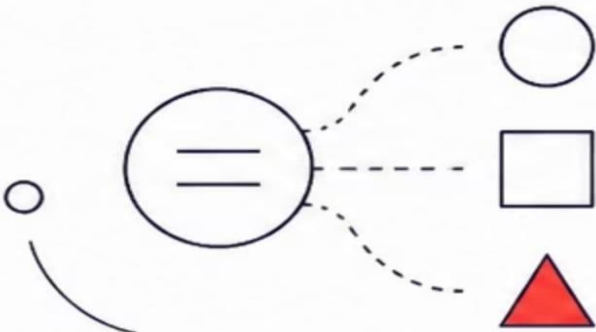
2. Object

(instance of a class)



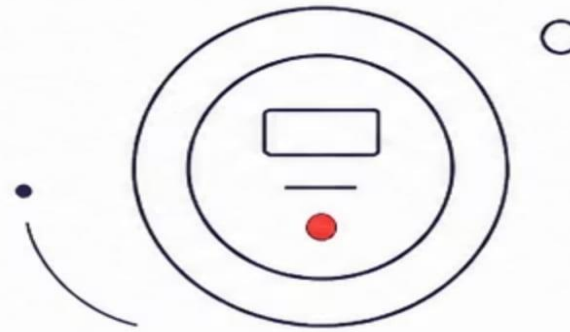
3. Inheritance

(acquiring properties from another class)



4. Polymorphism

(one interface, many forms)



Encapsulation

(binding data and methods together)



Abstraction

(hiding implementation details)

Class & Object

```
class Student:
    def display(self):
        print("Welcome")

s = Student()
s.display() # Output: Welcome
```

A class is a blueprint; an object is a specific instance of that blueprint (e.g., Student1, Student2).

Constructor `__init__`

```
class Student:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)

s = Student("Alice")
s.show() # Output: Alice
```

The constructor runs automatically when an object is created, initializing its attributes.

Encapsulation — Binding Data & Methods

```
class Account:  
    def __init__(self):  
        self.balance = 1000
```

Bundles data and methods within a class, protecting data from unauthorized external access.

Inheritance — Reuse Parent Properties

```
class Animal:  
    def sound(self):  
        print("Animal Sound")  
  
class Dog(Animal):  
    pass  
  
d = Dog()  
d.sound() # Animal Sound
```

Dog inherits all methods from Animal without rewriting them.

Polymorphism — One Interface, Many Fe Abstraction — Hide Implementation

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

animals = [Dog(), Cat()]
for a in animals:
    a.sound()
# Bark
# Meow
```

```
from abc import ABC

class Shape(ABC):
    pass
```

Abstraction shows only essential details while hiding internal implementation. It is widely used in large-scale software systems to enforce consistent interfaces across different classes.

Real-World OOP: Banking System



Class

BankAccount —
the blueprint
defining all
account behavior.

Objects

Account1,
Account2,
Account3 —
individual
instances for each
customer.

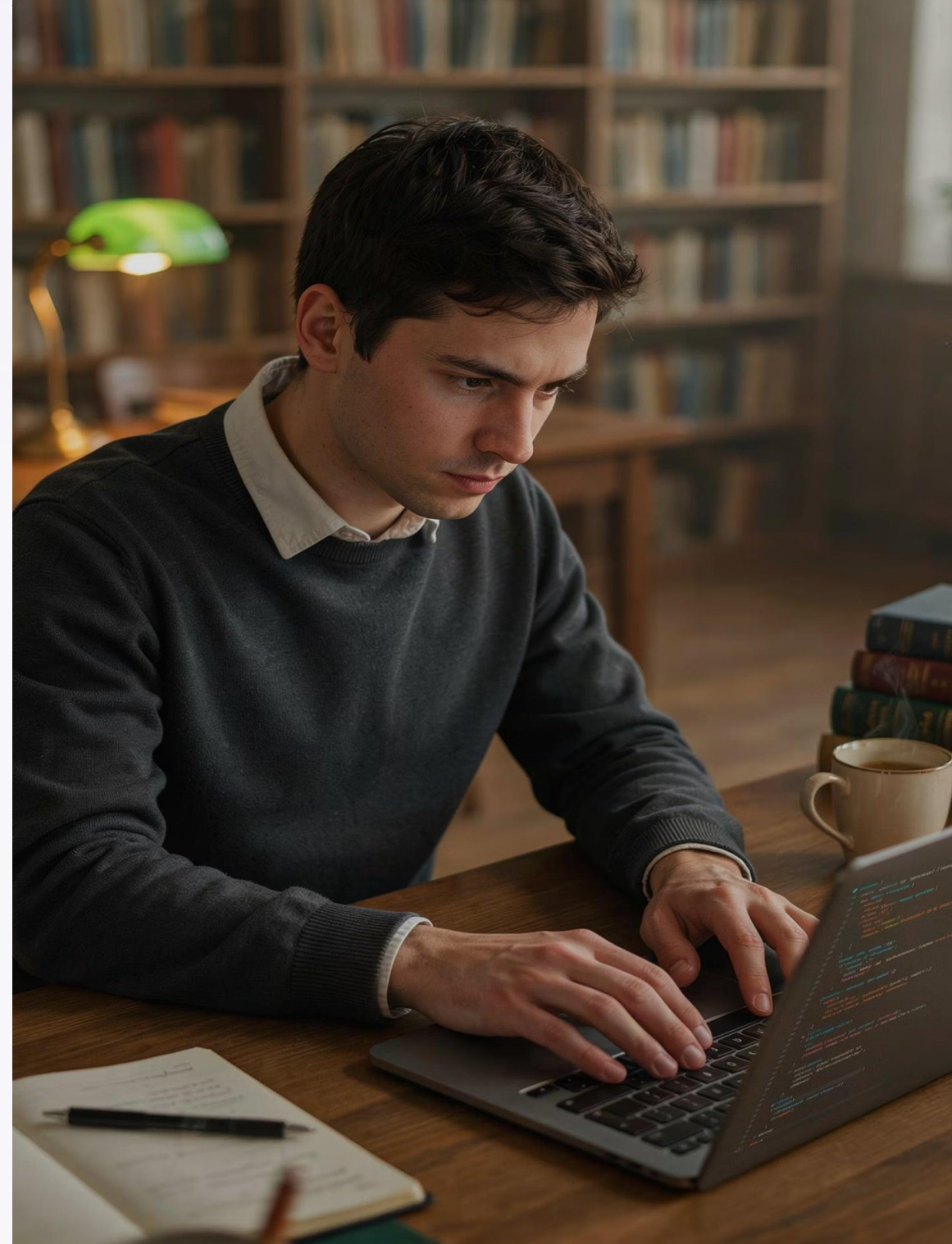
Methods

Deposit(), Withdraw(), CheckBalance()
— actions each account can perform.

Summary: Three Pillars of This Module

Concept	Purpose	Key Topics
String Methods	Text Processing	upper, lower, split, join, replace, find, f-strings
Exception Handling	Error Management	try, except, else, finally, multiple exceptions
OOP	Software Design	Classes, Objects, Constructors, Encapsulation, Inheritance, Polymorphism, Abstraction

📱 All three concepts work together in real Python applications — strings are processed inside OOP methods, and exceptions guard against runtime errors throughout.



1

String Operations

Write a program to convert a string to uppercase and count the number of vowels.

2

Exception Handling

Handle a division-by-zero scenario using a try-except block with a user-friendly message.

3

Student Class

Create a Student class with attributes for Name and Roll Number, and a method to display details.

4

Inheritance

Implement inheritance using a Person parent class and an Employee child class that extends it.

- **String Handling Fundamentals**
 - Strings and their characteristics
 - String indexing and slicing
 - String immutability and iterability
- **String Handling Methods**
 - upper(), lower(), title()
 - strip(), replace()
 - split(), join()
 - find() and count()
 - String formatting using f-strings
- **Exception Handling in Python**
 - Runtime errors and exceptions
 - Common exception types
 - try, except, else, and finally blocks
 - Handling multiple exceptions
 - Exception handling workflow
- **Object-Oriented Programming (OOP) Fundamentals**
 - Introduction to OOP
 - Classes and objects
 - Constructors (__init__ method)
- **Core OOP Concepts**
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
- **Real-World Applications of OOP**
 - Banking system example
 - Benefits of modularity, reusability, and scalability
- **Integration of Strings, Exceptions, and OOP**
 - Using string processing within classes
 - Handling runtime errors in object-oriented programs

Reference

Alvaro Fuentes, Become a Python Data Analyst – By Packt Publishing (2018)

Bharti Motwani, Data Analytics using Python – By Wiley (2020)

Jules S. Damji, Learning Spark: Lightning-Fast Data Analytics, Second Edition – By Shroff/O'Reilly (2020)

Data Science and Machine Learning using Python – 10 August 2022, MGH, 2022